

Introduction to Software Architecture

5 Pillars of Software Architecture

The five major pillars of software architecture—Robustness, Scalability, Maintainability, Security, and Performance—form the foundation of a well-designed system.

- Robustness ensures the system can handle failures gracefully, recover from errors, and provide reliable service under varying conditions.
- Maintainability focuses on designing a system that is easy to update, debug, and extend, ensuring long-term adaptability to evolving requirements.
- Performance ensures the system operates efficiently, delivering fast responses and optimizing resource usage to meet user expectations. Together, these pillars create a resilient, efficient, and future-proof software system.
- Scalability addresses the system's ability to grow seamlessly with increasing demands, maintaining performance and availability.
- Security protects the system from threats by enforcing data privacy, authentication, and authorization, safeguarding sensitive information.

Robustness

- Design for failure recovery: Implement retry mechanisms, circuit breakers, and fallback options.
- Use idempotency in operations: Ensure repeated requests have the same result.
- Ensure fault isolation: Contain the impact of failures within specific components.
- Implement graceful degradation: Provide reduced functionality during partial outages.
- Validate input and output rigorously to prevent unexpected errors.
- Include health checks: Regularly monitor the health of services and dependencies.
- Employ chaos testing: Simulate failures to ensure the system can handle them gracefully.
- Use redundancy: Duplicate critical components to avoid single points of failure.
- Monitor resource limits: Prevent resource exhaustion with quotas and throttling.

Maintainability

- Follow modular design: Create loosely coupled components with well-defined interfaces.
- Apply Domain-Driven Design (DDD): Align the system architecture with business domains.
- Ensure separation of concerns: Keep logic for different functionalities separate.
- Use standardized code practices: Implement consistent naming conventions and clean coding principles.
- Write comprehensive unit tests, integration tests, and end-to-end tests.
- Maintain clear and detailed documentation for APIs, architecture, and code.
- Use version control: Follow proper branching strategies like Gitflow.
- Emphasize observability: Implement logging, monitoring, and distributed tracing.

Performance

- Optimize database queries: Use indexes, denormalization, and avoid N+1 query problems.
- Use CDN (Content Delivery Network): Distribute static assets closer to the users.
- Minimize network latency: Implement compression, minimize API payloads, and reduce HTTP requests.
- Profile and optimize critical code paths: Use profilers to identify bottlenecks.
- Employ lazy loading: Load resources only when necessary.
- Leverage parallel processing: Execute tasks concurrently where possible.
- Use connection pooling for database and network connections.
- Optimize resource usage: Right-size instances and containers to minimize overprovisioning.

Scalability

- Design for horizontal scaling: Use stateless components and shared-nothing architecture.
- Leverage microservices: Break down applications into smaller, independently scalable services.
- Implement asynchronous communication: Use message queues or event-driven architectures.
- Optimize database design for scaling: Use sharding, replication, and partitioning.
- Enable auto-scaling: Configure infrastructure to scale up/down automatically based on demand.
- Design APIs to support pagination, filtering, and sorting.
- Employ load balancers: Distribute traffic evenly across instances.

Secure

- Follow the principle of least privilege: Grant users and services only the access they need.
- Implement authentication and authorization: Use protocols like OAuth2 or OpenID Connect.
- Secure sensitive data with encryption: Encrypt data both at rest and in transit.
- Regularly apply security patches to keep dependencies up-to-date.
- Use input validation and sanitize data to prevent SQL injection, XSS, and other attacks.
- Implement rate limiting and throttling to mitigate DDoS attacks.
- Log and monitor security events: Detect and respond to breaches in real-time.
- Adopt secure coding practices: Avoid hardcoding credentials and secrets in code.
- Perform regular security audits: Conduct penetration testing and vulnerability assessments.
- Use firewalls and network segmentation: Protect internal services from external threats.
- Enable multi-factor authentication (MFA) for all critical access points.

Design Patterns

Design Patterns are proven, reusable solutions to common problems that software developers face when designing and building systems.

They are not code, but conceptual templates or blueprints that can be implemented in different ways, across various programming languages.

Key Points About Design Patterns

- They capture best practices from experienced developers.
- They help solve recurring design problems in software architecture.
- They promote code reusability, extensibility, and maintainability.
- They provide a common vocabulary for technical discussions.
- They speed up development by avoiding reinventing the wheel.

Hands On: Online Travel Booking System

Imagine a system where a user books a travel package, which includes:

- Flight Reservation
- Hotel Booking
- Car Rental

All these steps must happen one after another. But if any step fails, everything done so far should be undone.

Hands On: Payment Processing in an E-Commerce Platform

Your e-commerce platform needs to support multiple payment methods, including:

- Credit Card
- PayPal
- Cryptocurrency
- Mobile Wallet (e.g., Apple Pay, Google Pay)
- Bank Transfer

The platform must:

Handle different payment methods transparently.

Allow adding new payment methods without changing existing core logic.

Let the system decide which payment method to use at runtime based on the user's choice.

Homework

Configurable and Reusable State Machine for Document Workflow Management

Modern enterprise systems often manage various types of documents such as policies, contracts, research papers, and legal documents. Each document type follows a specific workflow lifecycle, defined by states like Draft, Review, Approved, Rejected, or Signed. While many of these states share common behaviors, the valid transitions between states vary based on the document type and organizational policies.

The Problem with Distributed System

In distributed systems, ensuring that all nodes in the network are in sync and always available to serve requests presents a fundamental challenge. When nodes are spread across multiple locations, network partitions can occur, causing communication breakdowns between them. To maintain operation during such partitions, systems must make a trade-off between consistency and availability. If a system prioritizes consistency, it may become unavailable to ensure all nodes have the same data. Conversely, if availability is prioritized, the system may serve outdated or inconsistent data. This inherent conflict makes it impossible to achieve both consistency and availability in the face of network partitions, leading to a necessary compromise in system design.

CAP Theorem

Proposed by Eric Brewer in 2000, CAP Theorem states that in a distributed data system, it's impossible to guarantee all three of the following properties at the same time:

- Consistency (C)
- Availability (A)
- Partition Tolerance (P)

Consistency

- Every read receives the most recent write or an error.
- Ensures that all nodes in a distributed system return the same data when queried.
- Example: If a user writes new data, all future reads should reflect this update immediately across all nodes.

Availability

- Every request (read or write) receives a response, though it might not be the most recent data.
- Prioritizes ensuring that the system remains operational, even at the expense of consistency.
- Example: Even if some nodes are slow or unresponsive, a response should still be provided from the available nodes.

Partition Tolerance

- The system continues to operate even if network failures (partitions) occur between nodes.
- Ensures that the system remains operational, even when communication between nodes is partially or fully disrupted.
- Example: If a network failure isolates some nodes, the system still serves requests.

The Trade-Off

Due to network partitions (a common reality in distributed systems), systems must typically choose between Consistency and Availability.

Therefore, in practical terms, distributed systems can achieve:

- CP (Consistency and Partition Tolerance) — sacrifices Availability.
- AP (Availability and Partition Tolerance) — sacrifices Consistency.
- CA is theoretically possible but impractical in distributed systems since Partition Tolerance is essential.

Eventual Consistency

Eventual consistency is a consistency model used in distributed computing where all replicas of a given piece of data will become consistent over time, assuming no new updates occur.

It's a form of weak consistency, allowing temporary data inconsistencies across replicas for improved availability and performance.

Principle

- The core principle of eventual consistency is that the system does not enforce immediate synchronization across replicas but ensures that they converge to the same state "eventually" without explicit synchronization.
- It's based on the idea that delays in communication are acceptable as long as data eventually aligns across all nodes.

Use Cases

- Often applied in distributed systems where high availability and partition tolerance (as per the CAP theorem) are prioritized over strict consistency.
- Commonly used in NoSQL databases like Cassandra, DynamoDB, and Riak, as well as in large-scale applications where immediate consistency is not critical, such as social media, e-commerce catalogs, and content delivery networks (CDNs).

Benefits

- **High Availability:** Allows the system to remain operational even if some replicas are temporarily out of sync, which is crucial in distributed environments with unreliable network connections.
- **Scalability:** Facilitates horizontal scaling by enabling replicas to operate independently, reducing the need for constant synchronization.
- **Performance:** Reduces latency by allowing updates to be processed quickly without waiting for all replicas to confirm changes.

Challenges

- Temporary Inconsistency: Users might read "stale" data, where different replicas show different versions of the data until consistency is eventually achieved.
- Complex Conflict Resolution: In scenarios with concurrent updates across replicas, conflicts can arise, requiring conflict resolution strategies to ensure consistent outcomes.
- Data Convergence Assurance: The system must guarantee that all replicas converge to the same state, even if network delays or failures occur.

Consistency Window

- Refers to the time frame in which replicas may be inconsistent after an update. The system should aim to minimize this window to ensure data convergence as quickly as possible.
- The duration depends on factors such as network speed, replication protocols, and system load.

Common Implementation Techniques

- Replication: Data is asynchronously replicated across nodes, with updates propagated to other replicas over time.
- Vector Clocks or Timestamps: Used to track the order of updates and detect conflicts, allowing the system to identify and resolve conflicting versions of data.
- Conflict Resolution Strategies: Techniques like “last write wins,” merge functions, or user-defined rules can be used to determine the final, consistent state in case of conflicts.
- Anti-Entropy Protocols: Mechanisms such as Merkle trees or gossip protocols help detect and reconcile differences across replicas to ensure eventual consistency.